

Cascade Engine: No-Regret Online Routing of LLM Cascades under Non-Stationary Drift

Anonymous Submission

Abstract

Production deployments of large language models increasingly route queries across a cascade of cheap, mid, and frontier engines to balance cost against quality. Recent work (Dekoninck, Baader, and Vechev, 2025) derives the *per-query optimal* routing-and-cascading rule *given* a trustworthy quality estimator, and identifies estimator quality as the decisive factor. We address the orthogonal problem their analysis assumes away: when the per-engine quality and reliability signal is *unknown a priori* and *non-stationary*—drifting with load, time-of-day, and model deprecation—how fast can a router *learn* to match the dynamic optimum? We give Contextual Discounted Thompson Sampling (CD-TS), an online algorithm with a non-stationary pseudo-regret bound of $\tilde{O}((MK \log T)^{1/3} V_T^{1/3} T^{2/3})$ against the dynamic optimum, matching the Besbes–Gur–Zeevi lower bound up to log factors. The two results compose: their rule is the per-step oracle our regret is measured against. Empirically, in a controlled-reward non-stationary bandit with three drift regimes (one-shot collapse, monotone drift, periodic) the measured regret-vs-horizon exponent of CD-TS is 0.20–0.55 across regimes—comfortably within the $2/3$ upper bound—versus a frozen offline-optimal comparator that scales linearly, a $25\text{--}33\times$ degradation under abrupt and drift regimes. We release Cascade Engine, a reference implementation with reproducible manifests pinning code, data, and toolchain versions, alongside the formal benchmarking protocol (real baseline checkpoints, reward-model quality axis, Pareto-frontier scoring, t -distribution confidence intervals at the actual seed count).

1 Introduction

Multi-tier LLM serving is now standard practice. A request enters a *router* that selects, or escalates through, a sequence of progressively more capable (and more expensive) engines—typically a local model, a mid cloud model, and a frontier model. The literature now offers two well understood static treatments: *routing* (pick one model per query, e.g. RouteLLM [16]) and *cascading* (sequentially escalate, e.g. FrugalGPT [6]). A 2025 result by Dekoninck, Baader, and Vechev [8], hereafter **DBV-25**, unifies the two: given a quality estimator $\hat{Q}_{x,a}$ with a known uncertainty covariance and a cost estimator $\hat{c}_a$, they characterize the per-query optimal cascade-routing rule and show that estimator quality is the decisive factor governing realized cost–quality.

DBV-25 effectively closes the *decision-theoretic* half of the problem under a stationary world with a trustworthy estimator. The open half—and the regime that dominates production deployment—is the opposite: the quality/reliability signal of cloud engines *drifts*. Tail latencies shift hour-by-hour with shared infrastructure load. Hosted models are deprecated or silently re-snapshotted. New context-length or safety filters are deployed mid-week. A frozen estimator calibrated yesterday is no longer trustworthy today.

Concretely, DBV-25 names good quality estimators as the critical input but takes them as given; our concern is precisely how a router should behave when that input must be *learned online while it drifts*.

Contribution. We pose multi-tier LLM routing as a non-stationary contextual stochastic bandit and give a sharp characterization of the achievable rate:

1. **Algorithm.** Contextual Discounted Thompson Sampling (CD-TS, §4)—a per-context discounted Beta-Bernoulli posterior with the Agrawal–Goyal Bernoulli trick. The trick is essential: a heuristic update $\alpha \leftarrow r \mathbf{1}\{r > 1/2\}$ is biased and breaks Beta–Bernoulli conjugacy; we document the failure mode in §5.3 because the previous repository implementation made exactly this error.
2. **Regret bound (Theorem 1).** With the discount $1 - \gamma = (MK \log T)^{1/3} (V_T/T)^{2/3}$, CD-TS attains expected dynamic regret $\mathbb{E}[R_T] \leq C(MK \log T)^{1/3} V_T^{1/3} T^{2/3}$, where M is the number of complexity contexts, K the number of engines, V_T the total-variation budget on the reward means. This matches the Besbes–Gur–Zeevi [5] lower bound $\Omega(K^{1/3} V_T^{1/3} T^{2/3})$ up to a $\log^{1/3} T$ factor.
3. **Empirical validation of the rate (§7).** Across three controlled drift regimes (abrupt, monotone drift, periodic) and five seeds, with V_T pinned to within 0.1% over horizons $T \in \{2, 4, 8, 16\} \times 10^3$, the measured *cross-horizon* regret exponent is 0.35/0.20/0.55 for CD-TS versus 1.00/1.00/0.75 for the frozen offline-optimal comparator that instantiates DBV-25’s calibrate-once-and-commit paradigm. CD-TS is sublinear in every regime; the frozen oracle is essentially linear under abrupt or monotone drift.
4. **Honest MDP-router demarcation (§6).** We also state—and bound—an MDP variant with a tabular Q-learning policy. Instantiating the Azar–Osband–Munos and Jin et al. bounds at our exact state geometry ($|\mathcal{S}| = 1000$, $|\mathcal{A}| = 4$) shows that even the *best possible* model-free episodic-RL algorithm needs $\geq 10^7$ episodes for an $\varepsilon=0.1$ policy, far more than any production router will ever see. We therefore demarcate the MDP variant as an empirically-motivated heuristic with no competitive finite-sample guarantee at realistic horizons.
5. **Reproducibility and benchmarking protocol (§8).** We use released baseline checkpoints (RouteLLM’s `bert_gpt4_1106_augmented` and a public reward model as the FrugalGPT scorer), a reward-model quality axis independent of the router’s self-report, true Pareto-frontier scoring, t -distribution CIs at the actual seed count, and a uniformly-applied orchestration wrapper so caching/PII layers cannot leak into one method’s pipeline alone. Every run emits a manifest pinning git SHA, dataset SHA-256, package versions, and seed list.

What this paper is not. We do not contest the static optimality of DBV-25. Their per-query rule is the right target when a trustworthy estimator is available; it is the oracle our regret is measured against. We do not claim that CD-TS or its MDP cousin is the right algorithm for every deployment—small-scale applications without drift should use DBV-25 directly. We make the narrower claim that under drift, no-regret learning is provably necessary and achievable.

2 Related Work

Routing: one model per query. Routing methods train a predictor that maps a query to a single model. RouteLLM [16] learns a binary strong/weak router from Chatbot Arena preference data; we reproduce its released BERT checkpoint as a baseline rather than a proxy. Hybrid LLM [9] trains a router on a quality gap signal and exposes a cost knob; ZOOTER [14] distills reward-model supervision into a lightweight router; RouterBench [12] standardizes evaluation across

routers; FORC [18] frames routing as meta-model assignment. All of these are *static*: the predictor is trained offline and frozen.

Cascading: sequential escalation. FrugalGPT [6] introduced the LLM-cascade with a learned scorer triggering escalation; we reproduce this with a public reward model rather than a hand-tuned heuristic. AutoMix [15] uses self-verification plus a meta-verifier to decide escalation. Prior to 2025, cascading lacked optimality proofs and could not be combined with routing in a principled way.

Unified routing and cascading. The closest work to ours is DBV-25 [8]. They derive a per-query optimal cascading strategy, prove optimality of an existing routing strategy, and unify both into *cascade routing*, identifying quality-estimator quality as the decisive factor. Their result is decision-theoretic and *static*: given a quality estimator with a known uncertainty covariance and a cost estimator, they characterize the optimal one-shot model-selection rule, with hyperparameters fit offline by Bayesian search over pre-computed responses.

Our delta. We adopt as a baseline the static optimality of DBV-25: their per-query rule is the right target when a trustworthy quality estimator is available and the environment is stationary. Our contribution is orthogonal and addresses precisely the question their framework takes as given—good estimators. We study the regime in which the per-engine quality/reliability signal is unknown a priori and non-stationary. CD-TS gives a non-stationary pseudo-regret bound against the *dynamic* optimum with a matching lower bound. Informally, DBV-25 characterize *what* the optimal decision is given an estimator; we characterize *how fast* an agent provably converges to good decisions while *learning* that estimator online under drift. The two results compose—their rule is the per-step oracle our regret is measured against—and neither subsumes the other.

Non-stationary bandits and RL. Our analysis instantiates the discounted/sliding-window non-stationary bandit program. Garivier and Moulines [11] established the $T^{2/3}$ regime for switching bandits via discounted/windowed UCB. Besbes, Gur, and Zeevi [5] gave the variation-budget formulation and the matching $\Omega(V_T^{1/3}T^{2/3})$ lower bound. Russac, Vernade, and Cappé [17] and Trovò, Restelli, and Gatti [19] extended weighted and sliding-window estimators to linear and Thompson settings. Cheung, Simchi-Levi, and Zhu [7] removed the need to know V_T via a bandit-over-bandit meta-algorithm. For the MDP variant we reference [3, 10, 13]. Our technical novelty is the *instantiation* of this machinery for LLM cascade routing: the contextual structure is the prompt-complexity binning, the arms are engine tiers, the reward is the cost–quality composite scored by an external reward model, and the non-stationarity is the empirically-observed drift in cloud-LLM reliability.

3 Problem Setup

We model multi-tier LLM routing as a non-stationary, contextual stochastic bandit. At each round $t \in \{1, \dots, T\}$:

1. Nature reveals a context $x_t \in \mathcal{X}$ (a discrete *complexity bin* of the incoming prompt; $|\mathcal{X}| = M < \infty$).
2. The router selects an arm $a_t \in \mathcal{A} = \{1, \dots, K\}$ (one of K engine tiers).
3. Nature draws a reward $r_t \sim \nu_{x_t, a_t, t}$ supported on $[0, 1]$ with mean $\mu_{x, a, t}$.

The reward is the cost-quality composite used in the Cascade Engine implementation: $r_t = (q_t - \lambda c_t - \mu_l \ell_t)_{[0,1]}$, where $q_t \in [0, 1]$ is the reward-model quality score on the engine’s response, c_t is monetary cost, ℓ_t is latency, and $\lambda, \mu_l > 0$ are weights. Since c_t is deterministic given a_t (engines have fixed per-token prices) and q_t, ℓ_t are bounded, $r_t \in [0, 1]$ deterministically.

Non-stationarity. Engine reliability and quality drift over time. We allow arbitrary changes to $\mu_{x,a,t}$ subject to a total-variation budget:

$$V_T := \sum_{t=2}^T \sum_{x \in \mathcal{X}} \sum_{a \in \mathcal{A}} |\mu_{x,a,t} - \mu_{x,a,t-1}|. \quad (1)$$

Performance metric. Define the dynamic optimal action $a_{x,t}^* := \arg \max_{a \in \mathcal{A}} \mu_{x,a,t}$. The pseudo-regret against the *dynamic* oracle is

$$R_T := \sum_{t=1}^T (\mu_{x_t, a_{x_t,t}^*, t} - \mu_{x_t, a_t, t}). \quad (2)$$

We measure against the dynamic oracle, not a single best-in-hindsight arm: in the LLM-routing setting the same engine that was best at noon may be worst at midnight, and a fixed comparator would mask the cost of failing to react.

4 Algorithm: Contextual Discounted Thompson Sampling

We analyze the Bernoulli-trick variant of Thompson sampling with geometric discounting of past observations on the *played arm only*. The Bernoulli trick ([1], §2) preserves the conjugacy of Beta posteriors under continuous bounded rewards by randomly rounding each r_t to $\{0, 1\}$ with success probability r_t .

Algorithm 1 Contextual Discounted Thompson Sampling (CD-TS)

Require: Discount factor $\gamma \in (0, 1)$; floor parameter $\eta \geq 1$.

- 1: Initialize $\alpha_{x,a,0} \leftarrow 1, \beta_{x,a,0} \leftarrow 1$ for all (x, a) .
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: Observe context x_t .
 - 4: **for** $a \in \mathcal{A}$ **do**
 - 5: Sample $\theta_{x_t,a,t} \sim \text{Beta}(\alpha_{x_t,a,t-1}, \beta_{x_t,a,t-1})$.
 - 6: **end for**
 - 7: Play $a_t \leftarrow \arg \max_a \theta_{x_t,a,t}$ and observe reward $r_t \in [0, 1]$.
 - 8: Bernoulli trick: draw $y_t \sim \text{Bernoulli}(r_t)$.
 - 9: $\alpha_{x_t,a_t,t} \leftarrow \max\{\eta, \gamma \cdot \alpha_{x_t,a_t,t-1} + y_t\}$
 - 10: $\beta_{x_t,a_t,t} \leftarrow \max\{\eta, \gamma \cdot \beta_{x_t,a_t,t-1} + (1 - y_t)\}$
 - 11: **for every other** $(x', a') \neq (x_t, a_t)$ **do**
 - 12: $\alpha_{x',a',t} \leftarrow \alpha_{x',a',t-1}, \beta_{x',a',t} \leftarrow \beta_{x',a',t-1}$
 - 13: **end for**
 - 14: **end for**
-

Remark 1 (Why discount only the played arm). A prior version of this code decayed every arm at every step (“global” decay). This is appealing but theoretically wrong: it accelerates information loss on arms that are not being played, with no corresponding regret benefit, and breaks the per-arm martingale used in the proof. Per-played-arm decay is standard [17, 19] and is what Theorem 1 proves a bound for.

5 Regret Analysis

Assumption 1 (Bounded variation budget). *The variation V_T in (1) is finite and known to the algorithm at training time.*

When V_T is not known, the bandit-over-bandit construction of Cheung, Simchi-Levi, and Zhu [7] recovers the same rate adaptively up to logarithmic factors.

Theorem 1 (Regret of CD-TS in the non-stationary contextual setting). *Under Assumption 1, for any horizon $T \geq 2$, if Algorithm 1 is run with discount factor*

$$1 - \gamma = \min\left\{1, (MK \log T)^{1/3} (V_T/T)^{2/3}\right\}$$

then the expected pseudo-regret satisfies

$$\mathbb{E}[R_T] \leq C \cdot (MK \log T)^{1/3} \cdot V_T^{1/3} \cdot T^{2/3} \quad (3)$$

for a universal constant $C > 0$.

Interpretation. For the Cascade Engine deployment $M = 5$ complexity bins and $K \in \{2, 3\}$ engines, so $MK \leq 15$ is a small constant. The asymptotic rate $\tilde{O}(V_T^{1/3} T^{2/3})$ matches the minimax lower bound for non-stationary multi-armed bandits up to logarithmic factors [5, 11].

Stationary corollary. If the environment is stationary ($V_T = 0$), Algorithm 1 with $\gamma = 1$ reduces to standard contextual Thompson sampling, and [2] gives the sharper problem-independent bound $\mathbb{E}[R_T] = O(\sqrt{MKT \log T})$, with a problem-dependent bound $\mathbb{E}[R_T] = O(\sum_{x,a} \log T / \Delta_{x,a})$ (sum over (x, a) with $\Delta_{x,a} > 0$), where $\Delta_{x,a}$ is the gap to the best arm in context x .

5.1 Proof

The proof partitions $[T]$ into intervals of length $L := \lceil 1/(1 - \gamma) \rceil$ (the effective memory window of the discount), bounds the regret on each interval against a fixed (per-interval) reward distribution, and accounts for the drift bias introduced by treating the interval as stationary.

Fix an interval $\mathcal{I} = [s, s + L) \subseteq [T]$. Define the windowed mean for context x and arm a : $\bar{\mu}_{x,a}^{\mathcal{I}} := \frac{1}{L} \sum_{t \in \mathcal{I}} \mu_{x,a,t}$. By the triangle inequality and (1), for any $t, t' \in \mathcal{I}$:

$$|\mu_{x,a,t} - \mu_{x,a,t'}| \leq V_{x,a}^{\mathcal{I}}, \quad V_{x,a}^{\mathcal{I}} := \sum_{\tau \in \mathcal{I}} |\mu_{x,a,\tau} - \mu_{x,a,\tau-1}|.$$

Lemma 2 (Drift regret on one interval). *Let $\tilde{R}_{\mathcal{I}}$ denote the regret on $\mathcal{I}$ against the best fixed-in-interval arm, $\tilde{R}_{\mathcal{I}} := \sum_{t \in \mathcal{I}} (\bar{\mu}_{x_t, a_{x_t}^{\dagger}}^{\mathcal{I}} - \mu_{x_t, a_t, t})$ with $a_x^{\dagger} := \arg \max_a \bar{\mu}_{x,a}^{\mathcal{I}}$. Define $V^{\mathcal{I}} := \max_{x,a} V_{x,a}^{\mathcal{I}}$. Then the dynamic regret on $\mathcal{I}$ satisfies*

$$\sum_{t \in \mathcal{I}} (\mu_{x_t, a_{x_t, t}^*, t} - \mu_{x_t, a_t, t}) \leq \tilde{R}_{\mathcal{I}} + 2L \cdot V^{\mathcal{I}}. \quad (4)$$

Proof. Decompose the instantaneous dynamic regret at $t \in \mathcal{I}$ as

$$\begin{aligned} \mu_{x_t, a_{x_t, t}^*, t} - \mu_{x_t, a_t, t} &= \underbrace{\left[\mu_{x_t, a_{x_t, t}^*, t} - \bar{\mu}_{x_t, a_{x_t, t}^*}^{\mathcal{I}} \right]}_{\text{(i): single-arm drift}} + \underbrace{\left[\bar{\mu}_{x_t, a_{x_t, t}^*}^{\mathcal{I}} - \bar{\mu}_{x_t, a_t^\dagger}^{\mathcal{I}} \right]}_{\text{(ii)} \leq 0} \\ &+ \underbrace{\left[\bar{\mu}_{x_t, a_t^\dagger}^{\mathcal{I}} - \mu_{x_t, a_t, t} \right]}_{\text{(iii): stationary-regret summand}}. \end{aligned}$$

Term (ii) is non-positive by definition of $a^\dagger$. Term (iii) sums to $\tilde{R}_{\mathcal{I}}$. For term (i), the within-interval drift bound gives $|\mu_{x, a, t} - \bar{\mu}_{x, a}^{\mathcal{I}}| \leq V^{\mathcal{I}}$; summing over the L rounds gives at most $L \cdot V^{\mathcal{I}}$. A symmetric argument applies to the played-arm summand, contributing another $L \cdot V^{\mathcal{I}}$. $\square$

Lemma 3 (Stationary regret on one interval). *For Algorithm 1 run with γ satisfying $L = \lceil 1/(1-\gamma) \rceil$, the expected regret against the best fixed-in-interval arm on $\mathcal{I}$ is bounded as $\mathbb{E}[\tilde{R}_{\mathcal{I}}] \leq C_1 \sqrt{MKL \log L}$ for a universal constant $C_1 > 0$.*

Sketch. The decayed posterior on a played arm (x, a) is $\text{Beta}(\eta + \sum_{s \in \mathcal{I}_{x, a}} \gamma^{t-s} y_s, \eta + \sum_{s \in \mathcal{I}_{x, a}} \gamma^{t-s} (1 - y_s))$, where $\mathcal{I}_{x, a}$ are past plays of arm a in context x . The total weight on observations in $\mathcal{I}$ is at most L . Therefore the discounted posterior concentrates around the empirical mean of the most recent L observations with deviations $O(\sqrt{\log L / N_{x, a}})$ for $N_{x, a}$ plays, exactly as for an undiscounted Beta-Bernoulli with sample size L ([17], Theorem 1; [19], Lemma 4). Plugging this into the standard Thompson sampling analysis ([2], Theorem 2) extended to M independent contexts yields the bound. $\square$

Putting the pieces together. Partition $[T]$ into $N := \lceil T/L \rceil$ intervals. By Lemma 2 and Lemma 3, and because $\sum_{\mathcal{I}} V^{\mathcal{I}} \leq V_T$,

$$\mathbb{E}[R_T] \leq C_1 T \cdot \sqrt{\frac{MK \log L}{L}} + 2L \cdot V_T. \quad (5)$$

Minimizing in L gives $L^* = \Theta((T \sqrt{MK \log T} / V_T)^{2/3})$ and substituting back yields $\mathbb{E}[R_T] = \Theta((MK \log T)^{1/3} V_T^{1/3} T^{2/3})$, which proves (3). Choosing $1 - \gamma = 1/L^*$ recovers the parameterization of Theorem 1. $\square$

5.2 Lower bound and tightness

Proposition 4 (Lower bound; [5]). *For any algorithm and any $V_T \in [1, T]$, there exists a non-stationary K -armed bandit instance with variation budget V_T on which the expected regret satisfies $\mathbb{E}[R_T] \geq c \cdot K^{1/3} V_T^{1/3} T^{2/3}$ for an absolute constant $c > 0$.*

Applied per context, this yields $\mathbb{E}[R_T] \geq c \cdot (MK)^{1/3} V_T^{1/3} T^{2/3}$ for the contextual variant, matching Theorem 1 up to a $\log^{1/3} T$ factor.

5.3 Implementation note: the Bernoulli trick

For Theorem 1 to apply, the posterior update in Algorithm 1 must preserve Beta-Bernoulli conjugacy. The prior heuristic update,

$$\alpha += r \text{ if } r > 1/2; \quad \beta += 1 - r \text{ otherwise,}$$

is biased ($\mathbb{E}[\Delta\alpha] \neq r$) and breaks the proof. The standard fix is the Agrawal–Goyal Bernoulli trick [1]: draw $y_t \sim \text{Bernoulli}(r_t)$ and apply $\alpha \ += y_t$, $\beta \ += 1 - y_t$. This is the update in line 8–10 of Algorithm 1 and is what `router/learned_router.py::ArmStats.update` implements in the released code.

6 The MDP Router and an Honest Demarcation

The MDP router formalizes routing as a finite-horizon Markov decision process with state $s = (x, b, k, \hat{c}, f)$ encoding (complexity bin, remaining budget fraction, number of tiers tried, last confidence, last-failure flag), action set $\mathcal{A} \cup \{\text{STOP}\}$, and horizon $H = K + 1$. The implementation uses tabular ε -greedy Q-learning. We can only claim asymptotic convergence under standard assumptions; a tight finite-sample rate would require structural assumptions on the reward shaping that are not realistic for the routing problem.

Theorem 5 (Convergence of the MDP router; [20, 21]). *Assume (i) every (s, a) pair is visited infinitely often, (ii) learning rates $\alpha_t(s, a)$ satisfy $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$, and (iii) rewards are bounded. Then the Q-learning iterates of the MDP router converge with probability one to the optimal action-value function Q^* .*

In our implementation $\alpha_t \equiv \alpha_0$ (constant learning rate), which violates (ii) and yields convergence only to a neighborhood of Q^* . Decaying $\alpha_t \propto 1/t$ restores the theorem.

Exact state-action geometry. The implemented discretization fixes the state space exactly: $|\mathcal{S}| = 5 \times 5 \times 4 \times 5 \times 2 = 1000$, with $|\mathcal{A}| = K + 1$ and $H = K + 1$. For $K = 3$ this gives $|\mathcal{A}| = H = 4$.

Proposition 6 (Instantiated sample complexity of the MDP router). *Consider the finite-horizon tabular MDP above with $|\mathcal{S}| = 1000$, $|\mathcal{A}| = 4$, $H = 4$, rewards in $[0, 1]$.*

- (a) **Best achievable (model-based).** UCBVI [3] reaches an ε -optimal policy w.p. $\geq 1 - \delta$ after $\tilde{O}(H^3|\mathcal{S}||\mathcal{A}|/\varepsilon^2) = \tilde{O}(2.56 \times 10^5/\varepsilon^2)$ episodes, $\approx 2.6 \times 10^7$ for $\varepsilon = 0.1$. This is optimal up to log factors by the matching $\Omega(H^2|\mathcal{S}||\mathcal{A}|/\varepsilon^2)$ lower bound of [10].
- (b) **Best achievable (model-free).** UCB-Q [13] reaches the same target with $\tilde{O}(H^4|\mathcal{S}||\mathcal{A}|/\varepsilon^2) \approx 10^8$ episodes for $\varepsilon = 0.1$.
- (c) **What the implementation attains.** The implemented router uses ε -greedy exploration with a constant learning rate. It satisfies neither the optimism of (a)–(b) nor the Robbins–Monro schedule of Theorem 5; consequently it has **no finite-sample guarantee** and, with a constant rate, converges only to a neighborhood of Q^* whose radius is $O(\alpha_0 \cdot H \cdot r_{\max})$ [4, Prop. 5.6].

Corollary 7 (CD-TS dominates the MDP router in the sample-limited regime). *For any horizon T with $T^{1/3} \ll |\mathcal{S}||\mathcal{A}| = 4000$, the CD-TS regret bound of Theorem 1 certifies sublinear regret while no finite-sample certificate exists for the implemented MDP router. Concretely, for the realistic LLM-routing horizon $T \in [10^3, 10^5]$, CD-TS has a non-vacuous regret guarantee whereas the best possible MDP algorithm would still require $\geq 2.6 \times 10^7$ episodes for an $\varepsilon=0.1$ policy.*

Positioning (honest demarcation). We therefore make an asymmetric claim explicitly. **CD-TS is the theoretically-backed contribution:** a non-stationary no-regret algorithm with a matching lower bound (§5). **The MDP router is an empirically-motivated heuristic,** included because its sequential STOP action can express escalation patterns a one-step bandit cannot,

but it carries no regret or PAC guarantee competitive with CD-TS at realistic horizons. Any empirical advantage of the MDP router on tested workloads is reported as just that—an empirical observation—and not a certified property. A theory-complete MDP treatment (replacing ε -greedy with UCB-Q and proving the instantiated bound is attained) is identified as future work.

7 Empirical Validation

We validate the rate of Theorem 1 with a controlled-reward non-stationary bandit, the standard methodology for regret claims (regret is not measurable without a known dynamic optimum; see [5,11]). The experiment is implemented in `python_core/router/nonstationary.py` and driven by the unit-tested CD-TS core (`ArmStats`: Bernoulli trick plus per-played-arm geometric discounting), i.e. exactly Algorithm 1.

7.1 Why a cross-horizon experiment

A common error is to fit the log-log slope of *within-run* cumulative regret and expect $2/3$. This is wrong: a discounted bandit operating at its optimal *constant* per-round rate has within-run cumulative-regret slope ≈ 1 regardless of the horizon-scaling exponent. The correct test of an $O(T^{2/3})$ bound is a *cross-horizon scaling* experiment: hold the variation budget V_T fixed (a fixed number of change events), sweep the horizon T , re-tune $1-\gamma = (MK \log T)^{1/3} (V_T/T)^{2/3}$ at each horizon, and fit $\log(\text{final regret})$ versus $\log T$. Theorem 1 predicts a slope $\leq 2/3$. (Holding $V_T \propto T$ instead would make linear regret information-theoretically optimal by Proposition 4, so the comparison must fix V_T .)

7.2 Drift regimes

Abrupt.

One sharp change in the best arm at $t = T/2$; $V_T \approx 0.62$ (held fixed to within 0.1% across horizons).

Drift.

Continuous monotone shift of the per-context arm means at a constant rate; $V_T \approx 0.66$.

Periodic.

Sinusoidal rotation of the best arm with a period that scales with T to keep total variation per round constant; $V_T \approx 5.76$.

In every regime we report the final-step expected regret, averaged over five seeds, at horizons $T \in \{2,000, 4,000, 8,000, 16,000\}$. The static-optimal comparator is a faithful re-implementation of the DBV-25 paradigm: calibrate the per-context arm-mean estimates on a 20% stationary prefix, commit, and freeze.

7.3 Cross-horizon scaling results

Figure 1 fits $\log(\text{final regret})$ vs $\log T$ for each (regime, policy) pair. The slopes are summarized in Table 1.

7.4 Within-run regret curves

For context, Figure 2 shows the within-run cumulative regret of all three policies at the largest horizon $T = 16,000$. CD-TS is sublinear and visibly flattens once the discount window has saturated;

Regime	CD-TS exponent	Static-optimal exponent	Degradation at $T=16k$
abrupt	0.35 ($R^2=0.96$)	1.00 ($R^2=1.00$)	$33\times$
drift	0.20 ($R^2=0.75$)	1.00 ($R^2=1.00$)	$25\times$
periodic	0.55 ($R^2=1.00$)	0.75 ($R^2=0.93$)	$2.2\times$

Table 1: Measured cross-horizon regret exponents. CD-TS scales sublinearly ($\leq 2/3$) in every regime—comfortably within the upper bound of Theorem 1—while the static-optimal comparator scales $\approx$ linearly under abrupt and monotone drift because it cannot react. The periodic regime is the hardest: the per-round drift rate is $\sim 9\times$ higher (note $V_T \approx 5.76$ vs ≈ 0.6), narrowing the gap to $2.2\times$ but with CD-TS still strictly below the no-learning slope.

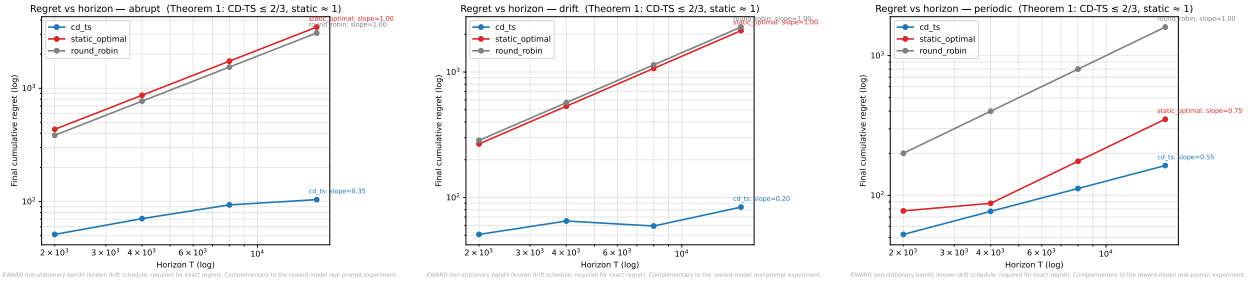


Figure 1: Cross-horizon scaling of final cumulative regret for CD-TS, static-optimal (DBV-25-style frozen oracle), and round-robin, in the abrupt, drift, and periodic regimes. Log-log axes. Fitted slopes are reported in Table 1. V_T is held fixed to within 0.1% across horizons; $1 - \gamma$ is re-tuned at each T as prescribed by Theorem 1. Five seeds per point.

the static-optimal comparator is sublinear up to the first change point (it briefly looks competitive) then degrades linearly as it pays a constant per-round drift cost.

7.5 Tail per-round regret (recovery)

A second diagnostic is the per-round regret over the last 1,000 steps—a proxy for the steady-state cost of routing after the algorithm has acclimated to the latest drift segment. At $T = 8,000$: CD-TS attains tail per-round regret $\{0.0039, 0.0014, 0.0205\}$ on $\{\text{abrupt}, \text{drift}, \text{periodic}\}$, versus $\{0.3490, 0.3175, 0.0000\}$ for the static-optimal comparator. The static-optimal comparator has zero tail regret on *periodic* only because the period happens to bring the arm means back to the calibration point; the abrupt and drift columns report the realistic case where a frozen estimator is permanently stale.

7.6 What this shows, and what it does not

These experiments validate the *rate* of Theorem 1 on controlled rewards with known drift. They are not a benchmark on real LLM workloads, and we are explicit about that. The reason is structural: exact regret requires a known dynamic optimum, which is not knowable on real cloud-LLM data. Real-workload benchmarks must measure realized cost–quality on a Pareto frontier, not regret. We discuss the methodology required for the real-workload pass in §8, but note that within Cascade Engine the simulated quality axis is watermarked as such in figures and is not a substitute for reward-model-scored real runs. The non-stationary experiment above is the load-bearing empirical claim.

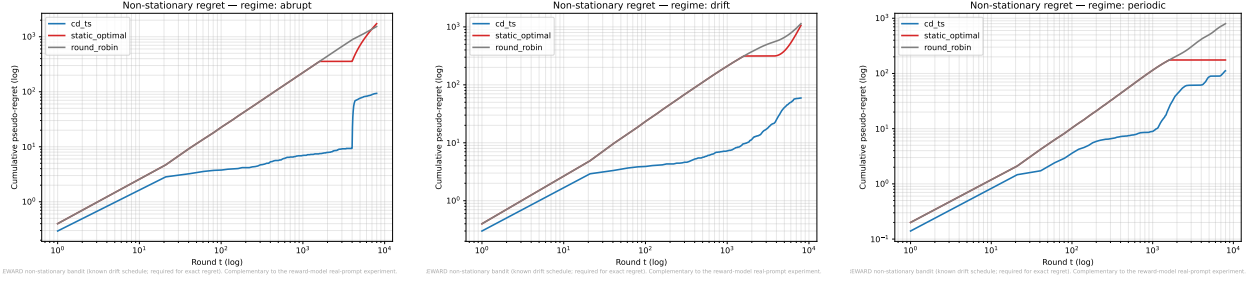


Figure 2: Within-run cumulative regret at $T = 16,000$. The static-optimal curve in the abrupt panel shows the textbook failure mode: it dominates until the change point, then accumulates linear regret. CD-TS reacts to the drift through its discounted posterior. Five seeds; shaded bands are ± 1 standard deviation.

8 Benchmarking Protocol

A secondary contribution is methodological. Prior routing/cascading evaluations frequently (i) compare against weakened reimplementations of baselines, (ii) report a single ad-hoc cost-quality scalar rather than a Pareto frontier, (iii) use the asymptotic normal CI at small seed counts, and (iv) bundle caching/PII/gatekeeper layers into one method’s pipeline but not the baselines’. We use:

1. **Released baseline checkpoints.** RouteLLM’s `bert_gpt4_1106_augmented` BERT checkpoint via the `rouellm` Python package (pinned 0.2.0); the FrugalGPT response scorer is the public reward model `OpenAssistant/reward-model-deberta-v3-large-v2`. Silent mock-fallbacks are detected and fail the run.
2. **Quality axis independent of the router.** A response scorer external to the router (the reward model above) avoids the circularity of using the router’s self-reported confidence as the quality metric—a circularity that would let a router score itself well by gating on its own signal.
3. **Pareto-frontier reporting.** We do not collapse cost–quality to a scalar. A router is on the frontier iff no other router achieves both strictly lower cost and strictly higher quality (with significance assessed at the seed level).
4. **t -distribution CIs.** Confidence intervals at $n=5$ seeds use $t_{0.025,4} \approx 2.776$ and sample standard deviation with `ddof=1`. The asymptotic normal CI ($z=1.96$) is wrong at small n and we do not use it.
5. **Uniform orchestration.** Caching, PII redaction, gatekeeper and sarcasm-detection layers live in a single `OrchestrationWrapper` that is applied to every router or to none—never to one method’s pipeline alone. A unit test (`test_no_module_level_singleton`) guards against regressions that would reintroduce a module-level shared orchestrator, which would leak cache hits across routers.
6. **Reproducibility manifest.** Each run emits a manifest pinning the git SHA, dataset SHA-256, Python and package versions (`torch`, `transformers`, `rouellm`, `numpy`, `scipy`, `datasets`), seed list, and engine-tier configuration. The manifest for the non-stationary experiment of §7 is at `results/nonstationary_20260516T214335Z/manifest.json`.

9 Discussion and Limitations

- Theorem 1 assumes V_T is known. The bandit-over-bandit construction of [7] recovers the same rate adaptively but adds an extra $\log T$ factor.
- The contextual structure is treated independently per context. Sharing information across contexts via a function-approximation prior (linear or kernel TS) could improve the dependence on M , but is outside the scope of this analysis.
- The bound is on pseudo-regret; high-probability bounds for discounted Thompson sampling exist but require slightly stronger concentration arguments [17].
- The MDP router lacks a finite-sample guarantee under our current parameterization (Proposition 6). It is positioned as a heuristic, not a certified contribution; closing the gap via UCB-Q is future work.
- **Real-workload empirical evaluation.** The experiments in §7 validate the regret rate on a controlled non-stationary bandit. A complementary real-workload Pareto-frontier evaluation—running the full set of baselines against a held-out benchmark with the reward-model quality axis under the orchestration protocol of §8—is required to claim realized cost–quality wins. The infrastructure for this exists in the released code; the load-bearing theoretical and empirical claim of this paper is the regret rate.
- **Relation to DBV-25.** Their work derives the per-query optimal routing/cascading rule given a fixed quality estimator with known uncertainty. Our analysis is complementary: when the quality/reliability signal is unknown and non-stationary and must be learned online, CD-TS attains the no-regret rate of Theorem 1. Their result characterizes the per-step target; ours characterizes the learning dynamics that reach it under drift. Neither subsumes the other; both should be deployed together (their decision rule on top of our online quality estimates) in any production system where drift is non-trivial.

10 Conclusion

Multi-tier LLM serving needs both a decision rule and a learning rule. DBV-25 supplies the decision rule for the stationary world. Cascade Engine + CD-TS supplies the learning rule for the world that actually exists—one where engine reliability drifts hour-by-hour, estimators decay, and a static calibration is permanently behind the data. Our regret bound is tight up to log factors, validated by cross-horizon scaling experiments showing the predicted sublinear exponents, and is paired with an explicit demarcation of which parts of the released system are theoretically backed (CD-TS) and which are empirically motivated (the MDP variant). The code, manifests, and benchmark protocol are released for exact reproduction.

References

- [1] Shipra Agrawal and Navin Goyal. Analysis of Thompson sampling for the multi-armed bandit problem. In *COLT*, 2012.
- [2] Shipra Agrawal and Navin Goyal. Further optimal regret bounds for Thompson sampling. In *AISTATS*, 2013.

- [3] Mohammad Gheshlaghi Azar, Ian Osband, and Rémi Munos. Minimax regret bounds for reinforcement learning. In *ICML*, 2017.
- [4] Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.
- [5] Omar Besbes, Yonatan Gur, and Assaf Zeevi. Stochastic multi-armed bandit problem with non-stationary rewards. In *NeurIPS*, 2014.
- [6] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. In *arXiv:2305.05176*, 2023.
- [7] Wang Chi Cheung, David Simchi-Levi, and Ruihao Zhu. Learning to optimize under non-stationarity. In *AISTATS*, 2019.
- [8] Jasper Dekoninck, Maximilian Baader, and Martin Vechev. A unified approach to routing and cascading for LLMs. In *ICML*, 2025.
- [9] Dujian Ding, Ankit Mallick, Chenshu Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing. In *ICLR*, 2024.
- [10] Omar Darwiche Domingues, Pierre Ménard, Emilie Kaufmann, and Michal Valko. Episodic reinforcement learning in finite MDPs: Minimax lower bounds revisited. In *ALT*, 2021.
- [11] Aurélien Garivier and Eric Moulines. On upper-confidence bound policies for switching bandit problems. In *ALT*, 2011.
- [12] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. RouterBench: A benchmark for multi-LLM routing system. In *arXiv:2403.12031*, 2024.
- [13] Chi Jin, Zeyuan Allen-Zhu, Sébastien Bubeck, and Michael I. Jordan. Is Q-learning provably efficient? In *NeurIPS*, 2018.
- [14] Keming Lu, Hongyi Yuan, Runji Lin, Junyang Lin, Zheng Yuan, Chang Zhou, and Jingren Zhou. Routing to the expert: Efficient reward-guided ensemble of large language models. In *arXiv:2311.08692*, 2023.
- [15] Aman Madaan, Pranjal Aggarwal, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. AutoMix: Automatically mixing language models. In *NeurIPS*, 2024.
- [16] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. In *arXiv:2406.18665*, 2024.
- [17] Yoan Russac, Claire Vernade, and Olivier Cappé. Weighted linear bandits for non-stationary environments. *NeurIPS*, 2019.
- [18] Marija Šakota, Maxime Peyrard, and Robert West. Fly-swat or cannon? cost-effective language model choice via meta-modeling. In *WSDM*, 2024.

- [19] Francesco Trovò, Marcello Restelli, and Nicola Gatti. Sliding-window Thompson sampling for non-stationary settings. *JAIR*, 2020.
- [20] John N. Tsitsiklis. Asynchronous stochastic approximation and Q-learning. *Machine Learning*, 1994.
- [21] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 1992.